

Searching Binary Embeddings

Calculations and Experiments

12 September 2026

Abstract

1. This report compares clustered scanning, Bitplanes and Backward Walk on one collection of binary documents. The calculations explain how cluster size, bit splits and key lookups affect work and memory.
2. Experiments check those estimates and measure when each method is faster under the same recall and RAM requirement.

1 Introduction

1.1 Exa method and end to end pipeline

1. **Source.** We follow Exa’s described search pipeline [1]; our implementation is method A.
2. **Before search.** Shorten Matryoshka embeddings, pack document signs, and assign documents to clusters.
3. **At query time.** Select nearby clusters, score their documents with the floating-point query, then rerank with uncompressed vectors.
4. **Scope.** B and C replace the blue search step below. Our timings measure this local implementation, not Exa’s production system.

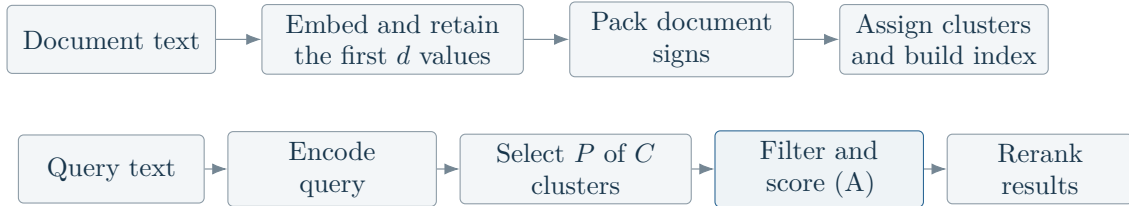


Figure 1: Exa’s described flow; the benchmark times cluster selection and search from cached embeddings.

1.2 Idea background

1. Use one five-value query to find the best possible document signs:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

2. A bit of 1 means +1; a bit of 0 means −1.
3. At bit 1, choose +1: $0.7 \times (+1) = 0.7$, rather than $0.7 \times (-1) = -0.7$.
4. At bit 3, choose −1: $-0.4 \times (-1) = 0.4$, rather than $-0.4 \times (+1) = -0.4$.
5. The preferred pattern is 11001. Its score is $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$.
6. A mismatch loses twice the absolute query value: 1.4 at the first bit, or 0.6 at the last. The full score is:

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

7. p sums the absolute query values at mismatched bits. Example: 11000 differs at the last bit, so its score is $2.5 - 2(0.3) = 1.9$.
8. Test whether B or C can find nearby patterns without scoring every document. The preferred pattern may be absent, so both methods keep alternatives.

1.3 Exploring two new methods: “Bitplanes” and “Backward Walk”

1.3.1 Bitplanes

1. Method B keeps the packed rows and adds a second arrangement of their signs. Each *bitplane* stores one bit position across all documents in a cluster. The figure shows three documents with the query from Section 1.2.
2. A candidate mask records which local rows remain.

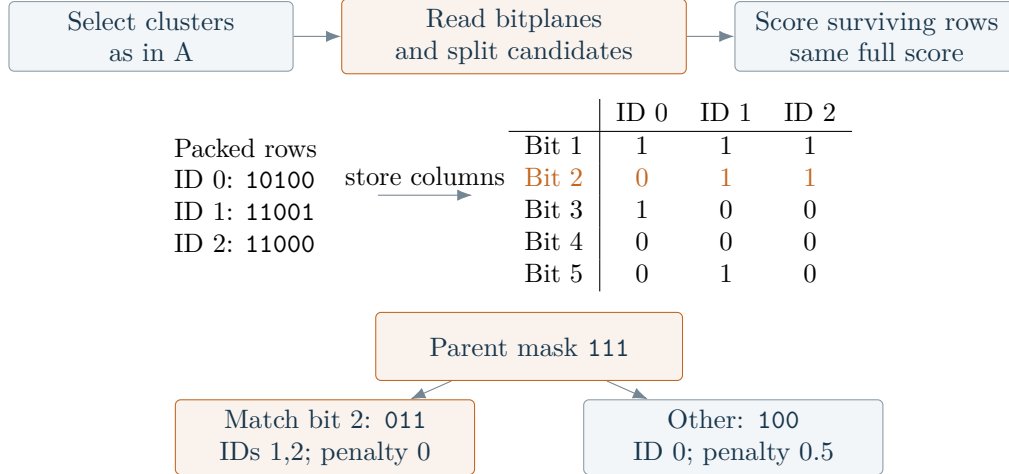


Figure 2: Orange marks the changed step; both child masks retain all three document positions.

3. The absolute sizes of the query values give this position order: 1, 4, 2, 3, 5. For a positive query value, a parent mask A is split into $A \& B_i$ and $A \& \neg B_i$.
4. The first child matches; the other adds $|q_i|$ to its penalty. Negative query values reverse the roles. Alternatives remain in a priority queue, ordered by known penalty, because a preferred early bit does not determine the final ranking.
5. Equation 1 gives a safe upper bound: a branch with known penalty p can score at most $Q - 2p$.
6. If IDs 1 and 2 score 2.5 and 1.9, the alternative with penalty 0.5 has bound 1.5 and cannot improve the top two. Equal-score cases retain the common document-ID tie rule.

```

bitplanes(query, clusters, K, leaf_size, node_budget):
    table, best = prepare_score_table(query), empty_top_k(K)
    order = positions_by_decreasing_abs_value(query)
    queue = full candidate mask for each selected cluster
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): break
        if node.count <= leaf_size or no_bits_remain(node):
            score_set_documents(table, node.mask, best)
        else:
            queue_nonempty_children(split(node, order))
    return best.sorted_by_score_then_id()

```

7. The leaf size decides when to stop splitting and score the remaining rows. Unlimited search with safe bounds is exact within opened clusters; a finite work limit can lose results.

1.3.2 Backward Walk

1. Method C forms a short key from a chosen set of h bit positions. In each cluster, documents with the same key are stored consecutively.
2. A directory maps an occupied key to the start and length of that row range. Search begins at the query's preferred key and enumerates alternatives in increasing weighted mismatch penalty.

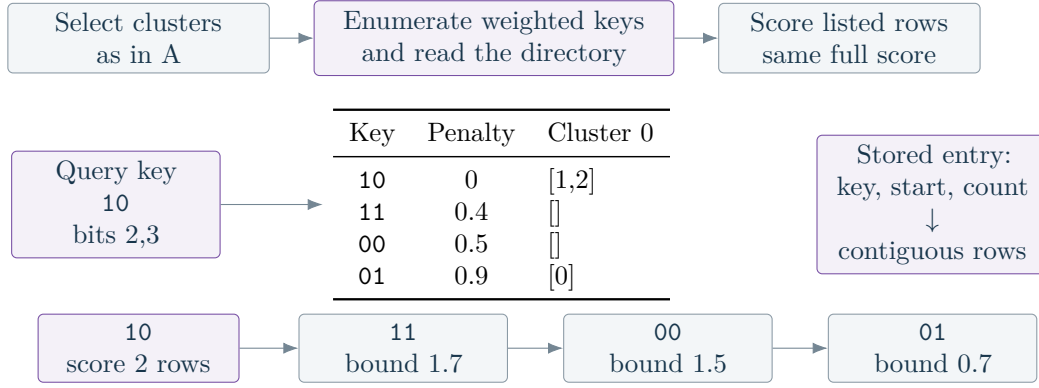


Figure 3: The first key returns IDs 1 and 2; the next key's bound of 1.7 permits stopping.

3. The ordering uses the sum of absolute query values at the flipped positions. Counting flipped bits alone would give the wrong order when query weights differ.
4. There is one shared enumeration queue across opened clusters, followed by a lookup in each cluster's directory. Empty lookups also take time.
5. Removing a key constraint is not a single ordinary hash lookup: the newly admitted patterns must be enumerated or supported by a different data structure.

```
backward_walk(query, clusters, K, key_positions, limits):
    table, best = prepare_score_table(query), empty_top_k(K)
    keys = unique_keys_by_increasing_penalty(query, key_positions)
    for key, penalty in keys:
        if cannot_improve(penalty, query, best): break
        for cluster in clusters:
            if work_limit_reached(limits):
                return best.sorted_by_score_then_id()
            rows = cluster.directory.lookup(key)
            score_each_row(table, rows, best)
    return best.sorted_by_score_then_id()
```

6. A short key omits some bit positions, so every retrieved row still receives the complete score. Finding enough candidates does not by itself prove sufficient recall.
7. Exhausting the keys or ruling out every remaining bound is exact within opened clusters; stopping at a candidate or lookup budget is approximate. Neither B nor C can recover a document excluded by the initial routing.

2 Preliminary calculations

- The collection contains $N = 1,000,000$ documents. The same 200 query vectors, 256-bit codes and reference answers are used throughout.
- Each method may choose its own cluster count C , probe count P and local settings. A probe opens one cluster.
- The result count is $K = 100$; d denotes the number of document bits.

2.1 Time complexity

- **One example for all three methods.** Open one cluster containing these three rows. Return the best two results: $N = 3$, $C = P = 1$, $d = 5$, $K = 2$.

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{ideal signs: } 11001.$$

ID	Document bits	Full score	Calculation
0	10100	0.1	$0.7 - 0.5 - 0.4 + 0.6 - 0.3$
1	11001	2.5	$0.7 + 0.5 + 0.4 + 0.6 + 0.3$
2	11000	1.9	$0.7 + 0.5 + 0.4 + 0.6 - 0.3$

- These are teaching rows, not a change to the benchmark dataset. Bits are shown in position order, left to right. Scores are rounded here.
- For larger collections, $n = N/C$ estimates rows per cluster if clusters are equal-sized.

2.1.1 Original method

Step 1: open the cluster and prepare the query.

- Open cluster 0: [ID 0, ID 1, ID 2]. One cluster lookup here.
- Following Exa's four-sign table method [1], split the five query values into $[0.7, 0.5, -0.4, -0.6]$ and $[0.3, 0, 0, 0]$.
- Each group has 16 sign patterns: $2 \times 16 = 32$ table entries. The final group is padded.
- The code checks four slots per entry: $32 \times 4 = 128$ slot visits. Table setup is $O(d)$ for a fixed group width of four. Cluster selection costs T_{route} .

Step 2: score each document.

$$\text{ID 0} \longrightarrow 0.1 \quad \text{ID 1} \longrightarrow 2.5 \quad \text{ID 2} \longrightarrow 1.9$$

- Loop over three rows; each reads two score-table entries: $3 \times 2 = 6$ terms. For F_A rows, this is $F_A \lceil d/4 \rceil$ terms, or $O(F_A d)$ work.

Step 3: keep the best two.

$$[0:0.1] \longrightarrow [1:2.5, 0:0.1] \longrightarrow \boxed{[1:2.5, 2:1.9]}.$$

- Offer three scores; keep at most two. The display above is sorted for readability. Keeping K results costs up to $O(F_A \log K)$; final sorting costs $O(K \log K)$.
- **Total.** One cluster lookup, 32 table entries built, six table terms read and three scores offered.
- These are different kinds of work, so we do not add them as equal CPU operations.

$$\boxed{T_A \approx T_0 + \frac{PN}{C} a_A.}$$

- T_0 includes cluster selection and table setup. a_A is average time per document, including its score and result update. For fixed C, P, d, K , scan work grows as $O(N)$.

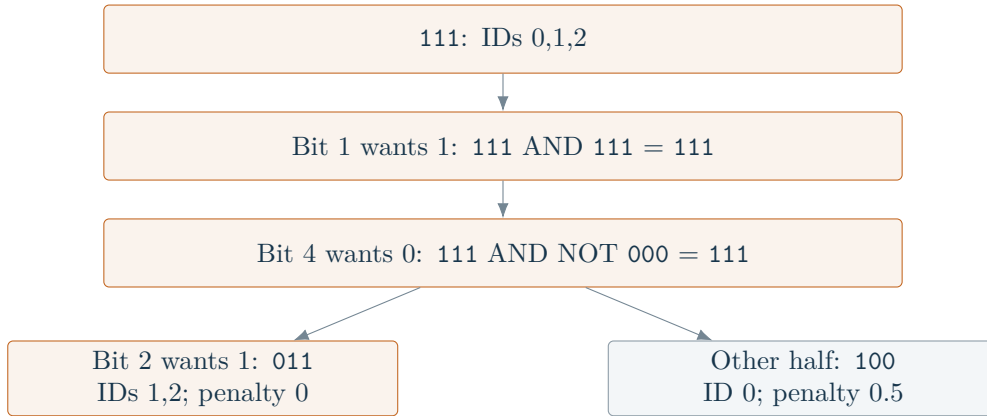
2.1.2 Bitplanes

- **Same input:** $q = [0.7, 0.5, -0.4, -0.6, 0.3]$; rows 10100, 11001, 11000; best two results. Stop splitting when at most two rows remain. Keep all other work limits unlimited.

Step 1: open, prepare and order the bits.

- Open the same cluster and build the same 32 score-table entries as A. Sort five query positions by absolute value: 1 (0.7) $\rightarrow$ 4 (0.6) $\rightarrow$ 2 (0.5) $\rightarrow$ 3 (0.4) $\rightarrow$ 5 (0.3).
- Sorting costs $O(d \log d)$. One pass sums the five absolute values: $Q = 2.5$, costing $O(d)$. Write one word for mask 111: all rows are active. Initial masks cost $O(PW)$ word writes.

Step 2: split the mask.



- The first two splits remove nothing. The third separates ID 0 from IDs 1 and 2. All three masks still have three positions. Each occupies one 64-bit word.
- Three splits visit $3 \times 1 = 3$ word positions. With J splits and mask width W , this costs $O(JW)$, where $W = \lceil n/64 \rceil$.

Step 3: score the two remaining rows.

- Mask 011 fits the leaf limit. Read one mask word, then score IDs 1 and 2. Two scores use $2 \times 2 = 4$ table terms. Best results: [1: 2.5, 2: 1.9].
- Reading L final masks costs $O(LW)$; scoring F_B rows costs $O(F_B d)$. Result updates have the same $O(F_B \log K)$ bound as A.

Step 4: check the waiting branch and stop.

- Its best possible score is $2.5 - 2(0.5) = 1.5$, below the retained score 1.9. Skip ID 0. One final bound check is $O(1)$ once the query sum and penalty are known.
- Four candidate sets were processed: three splits and one leaf. Including the P initial sets, queue work is bounded by $O((P + V) \log(P + V + 1))$.
- **Total.** Same table setup as A; sum five weights; sort five positions; write one initial mask; visit three split words and one final-mask word; score two rows; skip one row.
- Fewer scores alone do not prove lower time.

$$T_B \approx T_0 + PS \left\lceil \frac{N}{64C} \right\rceil b + f_B \frac{PN}{C} a_B + T_{\text{other},B}.$$

- S is average splits per opened cluster; f_B is the scored fraction; b is time per split-word visit. Here $S = 3$, $f_B = 2/3$. Other time includes sorting, queues and final-mask reads.
- A wide mask still costs time after many rows are removed.

2.1.3 Backward Walk

- **Same input:** $q = [0.7, 0.5, -0.4, -0.6, 0.3]$; rows 10100, 11001, 11000; best two results. Use bits 2 and 3 as the key: $h = 2$. Keep lookup and candidate limits unlimited.

Step 1: form the preferred key.

$$q_2 = +0.5 \rightarrow 1, \quad q_3 = -0.4 \rightarrow 0, \quad \boxed{\text{preferred key} = 10}.$$

- Open the same cluster and build the same 32 score-table entries as A. Read two signs: $O(h)$. Sort the two weights: $O(h \log h)$.
- One pass sums all five absolute query values: $Q = 2.5$, costing $O(d)$. Flipping bit 3 costs 0.4 in penalty; flipping bit 2 costs 0.5. Try the smaller penalty first.

Step 2: look up the preferred key.



- One lookup finds two rows. The other stored key, 01, contains ID 0. Score both rows with all five query values: four table terms, not just two key bits.
- H hash lookups cost expected $O(H)$ under normal hash-table behavior. Scoring costs $O(F_C d)$, plus $O(F_C \log K)$ result updates.

Step 3: check the next key and stop.

$$\underbrace{10}_{\text{score IDs 1,2}} \longrightarrow \underbrace{11}_{\text{penalty 0.4, bound } 2.5-0.8=1.7} \longrightarrow \boxed{1.7 < 1.9 : \text{stop}}.$$

- The next key cannot beat the current second result. Do not look it up. The remaining keys have larger penalties, so they cannot improve the result either.
- One key was looked up; one next-key state was queued. The final bound check is $O(1)$. Generating and ordering E visited keys costs up to $O(E \log(E + 1))$ after the initial sort.
- **Total.** Same table setup as A; sum five weights; sort two key positions; make one lookup; score two rows; check one next-key bound.
- All methods return IDs 1 and 2, so this example has 100% recall. It does not establish a speedup on real data.

$$\boxed{T_C \approx T_0 + P E c + F_C a_C + T_{\text{other}, C}.$$

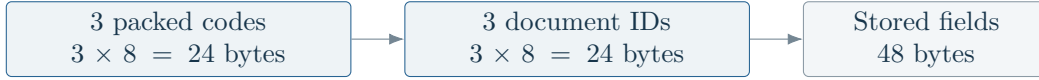
- E is keys tried in each opened cluster; c is time per lookup; F_C is rows scored. Other time includes key ordering and range starts. Use actual lookup count if a budget stops mid-key.
- Only for equal rows per key, $F_C \approx PEN/(C2^h)$. Here that estimate gives 3/4 of a row, while the actual key holds two. Count the stored rows for an exact answer.
- Wider keys can mean fewer rows per hit but more empty lookups. The required recall sets how far the search must continue.

2.2 Memory complexity

- **Same example:** three documents, five bits each, one cluster, two results. We count bytes of data first.
- Full process RAM also includes unused capacity, queue records, hash-table overhead, Python, and other runtime memory.

2.2.1 Original method

Step 1: store the codes and IDs before search.



- Each five-bit row uses one eight-byte word. Packing all 15 bits across row boundaries would use a different layout; this implementation stores whole words per row.
- Formula: $M_{\text{rows}} = N(8 \lceil d/64 \rceil + 8)$ bytes. Size is $O(N \lceil d/64 \rceil + N)$. Cluster data is extra. For example, C float32 centroids of width d_R use $4Cd_R$ bytes. This one-cluster example needs no centroid search.

Step 2: hold the query, score table and current results during search.

$$\boxed{5 \times 4 = 20} + \boxed{32 \times 8 = 256} + \boxed{2 \times (8 + 8) = 32} = 308 \text{ bytes.}$$

- Left to right: float32 query, double-precision table entries, result IDs and scores.
- The result buffer holds two entries even though A checks three documents. It does not need a stored score for every document.
- General sizes: $4d + 128 \lceil d/4 \rceil + 16K$ bytes, or $O(d + K)$ for these buffers.
- **Total counted fields:** 48 bytes stored + 308 query bytes = 356 bytes. This is not total process RAM; cluster records and allocation overhead remain additional.

2.2.2 Bitplanes

Step 1: keep A's rows and add the bitplanes.

	ID 0	ID 1	ID 2	Packed storage
Bit 1	1	1	1	8 bytes
Bit 2	0	1	1	8 bytes
Bit 3	1	0	0	8 bytes
Bit 4	0	0	0	8 bytes
Bit 5	0	1	0	8 bytes

- Each plane has three useful bits but occupies one word. Five planes add $5 \times 8 = 40$ bytes. Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{planes}} = 8d \sum_c \lceil n_c/64 \rceil$ bytes. Include padding in every cluster.

Step 2: hold masks while splitting.

$$\underbrace{111}_{\text{parent: 8 bytes}} \longrightarrow \underbrace{011}_{\text{child: 8 bytes}} + \underbrace{100}_{\text{child: 8 bytes}} .$$

- The parent exists while both children are built: $3 \times 8 = 24$ live mask bytes at the peak. All three masks keep their full width, even if one child is empty.
- Sorting five positions adds $5 \times 8 = 40$ bytes on this 64-bit machine. Counted query data: A's $308 + 24 + 40 = 372$ bytes. Queue records are still extra.
- For mask width $W_c = \lceil n_c/64 \rceil$ and $A_c(t)$ live masks, peak mask data is $8 \max_t \sum_c A_c(t) W_c$ bytes. Waiting branches count too.
- **Total counted fields:** 88 bytes stored + 372 query bytes = 460 bytes.
- Stored size is $O(N \lceil d/64 \rceil + N + d \sum_c W_c)$; query masks grow with the number and width of live branches, not just the number of remaining document IDs.

2.2.3 Backward Walk**Step 1: group rows by their two-bit key before search.**

Key	Stored rows	Start	Count
10	[ID 1, ID 2]	0	2
01	[ID 0]	2	1

- Keep one copy of the codes and IDs: 48 bytes. Rows with the same key sit together.
- Each entry holds a four-byte key, eight-byte start and eight-byte count: 20 bytes of fields. Two occupied keys add $2 \times 20 = 40$ bytes. Empty keys have no stored entry.
- Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{entries}} = U(4 + 8 + 8)$ for U occupied keys. Total stored size is $O(N \lceil d/64 \rceil + N + U)$, plus the hash table's allocation overhead.

Step 2: hold the key order and next key during search.

$$\underbrace{[3, 2]}_{\text{two positions: 16 bytes}} \quad \underbrace{11, \text{ penalty } 0.4}_{\text{one waiting state: 16 bytes}} .$$

- The waiting state stores an eight-byte penalty, four-byte flip mask and four-byte position. This example's measured key-queue peak is 16 bytes.
- Counted query data: A's $308 + 16 + 16 = 340$ bytes. Selected-cluster pointers and runtime objects are extra. With Q waiting states, the extra arrays take $O(h + Q)$ space.
- **Total counted fields:** 88 bytes stored + 340 query bytes = 428 bytes.

What these totals mean.

- The totals add the listed buffer sizes; their peaks need not occur together. They are not measured process RAM. Reserved space and unlisted objects use extra bytes.
- The 32 GB experiment limit applies to measured process RAM, including overhead. Storage is built once. Query buffers are temporary; concurrent queries need their own work space.

2.3 Recall estimation

- Reference IDs are the exact best results under the full binary score. In our small example they are $[1, 2]$. All methods return both: $2/2 = 100\%$ recall.
- In the benchmark, each query has $K = 100$ reference IDs. Returning 99 gives 99% recall. This checks the ranking, not whether the text answers a person's question.

- Let G_q be reference IDs and $O_{a,q}$ returned IDs. Across the same $n_q = 200$ queries,

$$\bar{R}_a = \frac{1}{n_q K} \sum_q |G_q \cap O_{a,q}|.$$

2.3.1 Original method

- Let g_q reference IDs be in the opened clusters. A scores all opened rows, so recall is g_q/K .

2.3.2 Bitplanes

- If B returns fraction $r_{B,q}$ of those IDs, recall is $(g_q/K)r_{B,q}$. For example, $(8/10)(6/8) = 60\%$. Safe, unlimited local search gives $r_{B,q} = 1$.

2.3.3 Backward Walk

- Likewise, recall is $(g_q/K)r_{C,q}$. Visiting or safely ruling out all keys gives $r_{C,q} = 1$. Limits can lose results. If $g_q = 0$, recall is zero.
- Average the per-query products; multiplying separately averaged fractions can give a wrong answer.

2.4 Summary

- Each formula uses its method's own opened clusters and local work.
- Let M_0 be allocated row and router storage, Q_0 common query memory, D_C allocated directory memory, and Q_B, Q_C extra masks and queues. Allocation overhead is included in these memory terms.

Time complexity

Method	Approximate query time
Original method	$T_0 + (PN/C)a_A$
Bitplanes	$T_0 + PS \lceil N/(64C) \rceil b + f_B(PN/C)a_B + T_{\text{other,B}}$
Backward Walk	$T_0 + PEc + F_C a_C + T_{\text{other,C}}$; for equal rows per key, $F_C \approx PEN/(C2^h)$

- The symbols and loop counts are defined in the steps above. Use actual cluster sizes and key counts for the measured comparison. Each cost belongs in one term only.

Memory complexity

Method	Stored index	Temporary query memory
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + D_C$	$Q_0 + Q_C$

- The process, including these arrays and the running program, must fit 32 GB. A peak that includes construction is reported as such, not as index size.

Recall estimation

Method	Per-query recall	Local exactness condition
Original method	g_q/K	Score all opened documents
Bitplanes	$(g_q/K)r_{B,q}$	Complete search with safe bounds
Backward Walk	$(g_q/K)r_{C,q}$	Visit or safely rule out remaining keys

- The reported batch value averages the same 200 reference answers.

3 Deriving Optima

1. Minimize retrieval time at the required recall, within 32 GB RAM.
2. Keep documents, queries and reference answers unchanged. Each method chooses its own index and search settings.

3.1 Original method

1. For each reference ID, record when its cluster is opened. Example: a document in the fifth cluster is included when $P \geq 5$.
2. Write that rank as $s_{qi}^{(C)}$ for result i of query q . The smallest probe count meeting recall ρ is:

$$P_A(C, \rho) = \min \left\{ P : \frac{1}{n_q K} \sum_{q=1}^{n_q} \sum_{i=1}^K \mathbf{1}[s_{qi}^{(C)} \leq P] \geq \rho \right\}.$$

3. The indicator counts IDs whose clusters are open. At 99% recall, 200×100 reference IDs require at least 19,800 matches. Compute these ranks before timing.
4. Try cluster counts 1, 256, 4096 and 8192. For each, test probe cutoffs for 80%, 90%, 95% and 99% recall, plus opening every cluster.
5. Measure time and rows scored. The cutoffs apply to this query batch; they do not guarantee recall for new queries.
6. For a simple derivative, assume equal clusters, routing cost αC , row cost β , and fixed P :

$$T_A(C) \approx \alpha C + \beta NP/C, \quad \frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2}, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}.$$

7. C^* balances cluster selection against document scanning. At a fixed recall target, $P_A(C, \rho)$ may change with C , so the measured comparison uses actual probe counts and cluster sizes.

3.2 Bitplanes

1. A 6400-document mask has 100 words. Removing 100 of 200 remaining candidates saves 100 scores, but adds 100 split-word visits.
2. Let $W = \lceil n/64 \rceil$. If a split avoids ΔF scores, it helps when:

$$\boxed{\Delta F a_B > Wb + \Delta T_{\text{other},B}.}$$

3. Left: saved scoring time. Right: split time and extra branch work. This assumes roughly constant cost per score; memory access and recall still need checking.
4. If each cut halves one set, j cuts leave about $n2^{-j}$ rows. Holding other work fixed gives:

$$T_B(j) \approx T_0 + jWb + n2^{-j}a_B + T_{\text{other},B}, \quad j^* = \log_2 \left(\frac{n \ln(2) a_B}{Wb} \right).$$

5. This estimates a cost balance, not recall. Identical bitplanes may remove no more rows; alternative branches add work. Measure both effects.
6. Try every cluster/probe option from A. Test leaf size 128 with budgets 256 and 1024; leaf sizes 512 and N with unlimited work; and leaves 32 and 128 with budget 4096.
7. A node budget counts visited sets, including leaves. A leaf of size N allows scoring a cluster immediately. Record splits, final-mask reads and scores.
8. A's probe cutoff is necessary for the same layout, but B's local limits may lose more IDs. Check the returned IDs against the recall target.

3.3 Backward Walk

1. For the first saved query, the ideal score is 12.361. Even after all eight key bits mismatch, the bound is 11.373, above the 100th score of 9.994.
2. That key cannot be discarded. All 200 queries have this issue for widths 1, 4 and 8.
3. At the same routing settings, a change from width h to h' helps when:

$$\boxed{\text{saved scoring time} > \text{extra lookup time} + \text{extra key-management time.}}$$

4. Saving 3 ms of scoring pays for 1 ms of extra key work. Saving 0.5 ms does not. These are cost examples, not measured outcomes.
5. A small first result set is not enough. More keys, including empty ones, may be needed to reach the recall target or rule out the remaining results.
6. Test widths 1, 4, 8, 16, 20 and 24. Short keys visit their full key spaces here; longer keys use lookup limits of 4096 and 65,536.
7. Keep the full 256-bit score. Measure directory size and returned IDs; do not infer recall or a winning width from $N/2^h$.

3.4 Comparison

1. Let Θ_a be method a 's tested settings. Choose its lowest measured time subject to the recall and memory limits:

$$\theta_a^* = \underset{\theta \in \Theta_a}{\operatorname{argmin}} T_{a,\text{measured}}(\theta) \quad \text{subject to} \quad \bar{R}_a(\theta) \geq \rho, \quad M_{a,\text{process}}(\theta) \leq 32 \times 10^9 \text{ bytes.}$$

Method	Adjustable settings	Work that may be reduced
Original method	Cluster count C and probes P	Documents reaching the scorer
Bitplanes	C , P , leaf size and node budget	Scores, at the cost of mask and branch work
Backward Walk	C , P , key width h and lookup limit	Scores, at the cost of lookups and range starts

2. Compare methods after each chooses its own qualifying setting. Keep results below the recall target in the records, but do not count them as wins.
3. This finds the best observed setting in the tested range on the unchanged queries. It does not prove a global optimum.

4 Experiments

4.1 Setup

1. **Data.** One million MS MARCO documents [3], 200 queries, and existing Nomic embeddings [4]. Every run uses the same arrays, values and query rows.
2. **Score.** Documents have 256 sign bits; queries have 256 float values. The reference is the exact best 100 IDs. Returning 99 on average gives 99% recall.
3. **Settings.** Vary clusters, probes, B's leaf size and branch budget, and C's key width and lookup limit. Keep the full score unchanged.
4. **Probes.** For each cluster count, find where A reaches 80%, 90%, 95% and 99% recall. Include opening all clusters. Give A, B and C these same choices.
5. **Machine.** Apple M5 Max, 128 GiB physical RAM, one CPU thread, and a 32 GB limit per process.
6. **Runs.** Test 256 settings. Each searches all 200 queries twice. Repeat the fastest qualifying settings in three processes on those same queries.
7. **Time.** Report the median. Include cluster selection and search; exclude index building and embedding generation.
8. **Input check.** Compare file hashes before and after the runs. No input array changed.

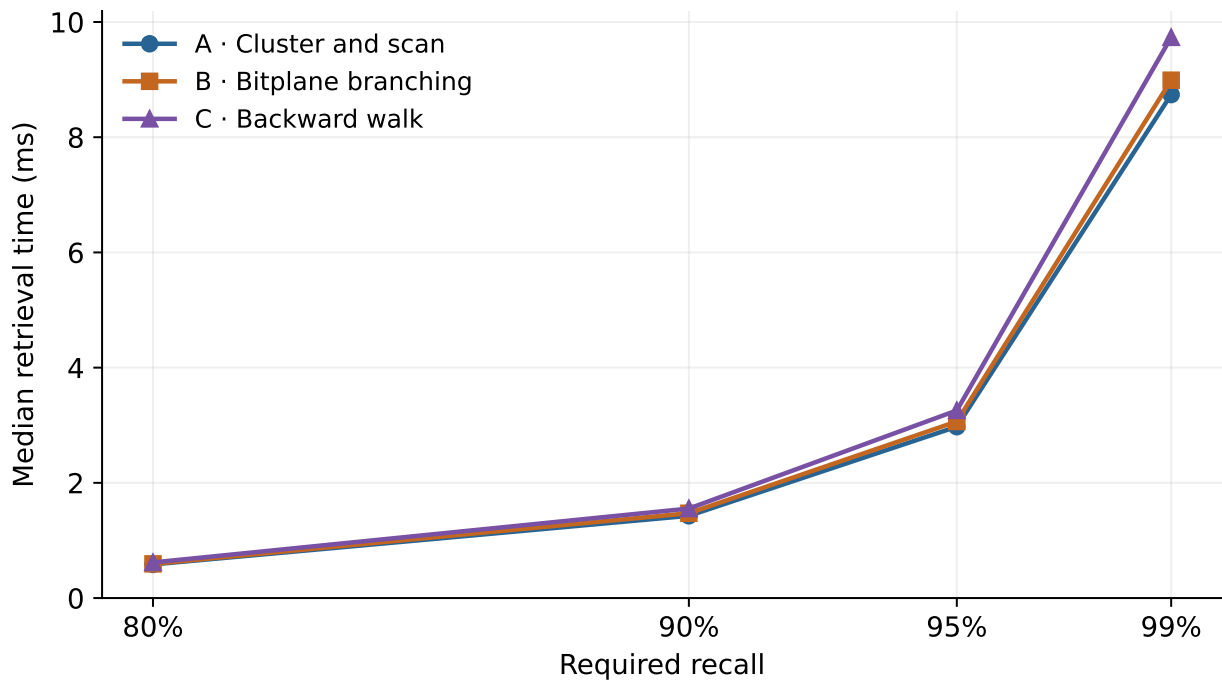


Figure 4: Median time at each recall target; every plotted setting meets the target on the same inputs.

4.2 Results and calculation

Table 1: Repeated measurements at the same recall requirements.

Required recall	Method	Actual recall (%)	Median (ms)
80%	A	80.01	0.5842
80%	B	80.01	0.5936
80%	C	80.01	0.6181
90%	A	90.02	1.43
90%	B	90.02	1.47
90%	C	90.02	1.55
95%	A	95.00	2.97
95%	B	95.00	3.07
95%	C	95.00	3.26
99%	A	99.00	8.74
99%	B	99.00	8.99
99%	C	99.00	9.73

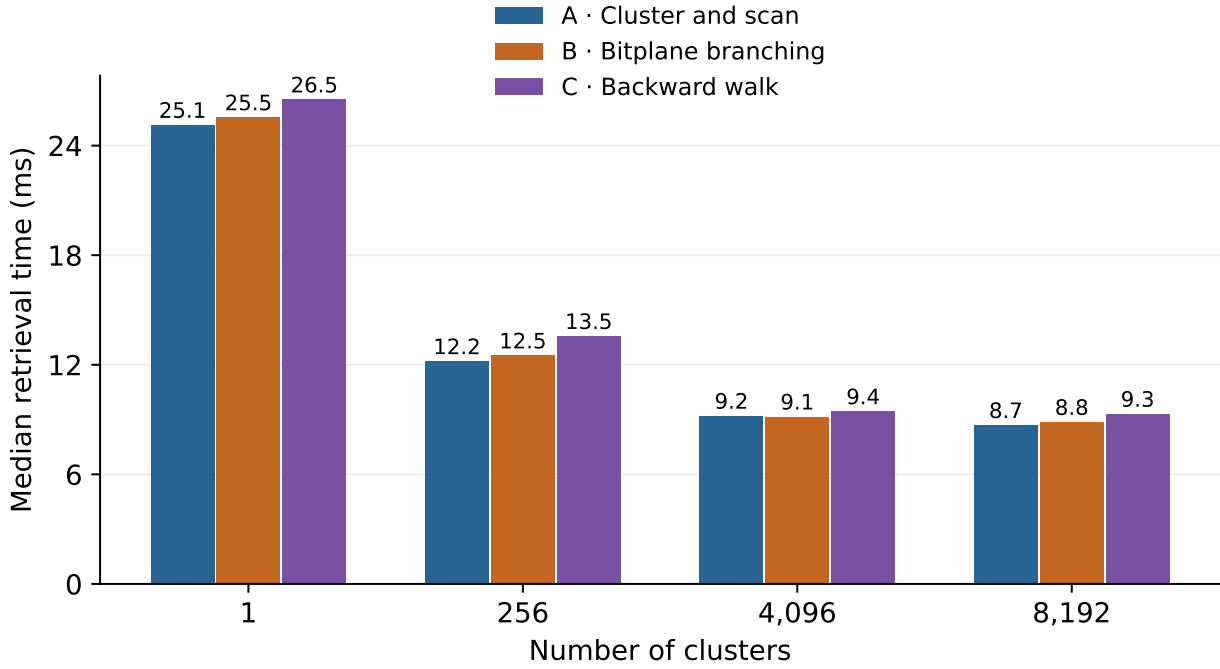


Figure 5: Initial 99% recall measurements by cluster count; Table 1 repeats selected settings, so small timing differences are expected.

Table 2: Selected settings for 99% recall.

Method	Route	C	P	L	Budget	h	RAM (MB)
A	IVF	8,192	2,440	—	—	—	190.6
B	IVF	8,192	2,440	512	0	—	231.6
C	IVF	8,192	2,440	—	0	1	161.7

1. L is leaf size; h is key width. Budget counts visited sets for B and lookups for C; zero means unlimited.
2. A large leaf can make B score a whole cluster without splitting. The next table shows the work actually performed.

4.3 What the numbers mean

Table 3: Average work per query for the selected 99% settings.

Method	Rows scored	Split words	Leaf words	Key lookups
A	295,858	0	0	0
B	295,858	0	5,820	0
C	295,858	0	0	4,880

1. At 99% recall, all three selected settings open 2440 of 8192 clusters and score 295,858 rows per query on average.
2. B makes no splits and reads 5820 mask words. C uses one-bit keys and makes $2440 \times 2 = 4880$ lookups. Neither avoids full scores at these settings.
3. Median times: A 8.74 ms, B 8.99 ms, C 9.73 ms.
4. Smaller leaves and larger branch budgets did not give a faster result at the same recall. The best 24-bit C result reached 73.8% recall within the tested limits.
5. A 20-bit C setting reached 95.315% recall in 29.71 ms. A took 2.97 ms at the 95% requirement.
6. Check stored codes, IDs, padded bitplanes and occupied key entries against the memory formula.
7. Measured process RAM also includes spare allocation capacity, temporary arrays and the running program.

Table 4: Index data calculated from the layout and reported by the implementation.

Method	Calculated (MB)	Index data (MB)	Peak RAM (MB)
A	48.454	48.454	190.6
B	88.765	88.765	231.6
C	48.762	48.762	161.7

8. For A, raising recall from 90% to 99% opens 2440 rather than 353 of 8192 clusters. Median time rises from 1.43 to 8.74 ms.
9. The queries are unchanged; opening more clusters adds work. The result applies to this dataset and the tested settings.

4.4 Code references

1. Run `experiments.fixed_data_comparison`: `sweep`, `extend`, then `repeat`. The README gives full commands and input paths.
2. Search code: `cpp/index.cpp` and `cpp/key_index.cpp`. All commands read the same benchmark arrays.
3. Records: `results/fixed-data-comparison-2026-09-12/`. Includes settings, input hashes, work counts and timings.

5 Limitations and Next steps

1. Only one unchanged document collection and query set were tested. Every method used the same score, reference IDs, RAM limit and CPU thread count.
2. The chosen settings were fastest within this tested range. They do not prove the best possible implementation.
3. Scores, mask reads and key lookups have different time costs. Memory access, code and hardware affect those costs.
4. The ideal bit pattern does not tell us how many branches or keys are needed. Check those counts on the actual data.
5. Next: expand the parameter range or improve a measured expensive operation on the same benchmark. Report any later change of model, documents or queries as a separate comparison.

6 Conclusion

1. A had the lowest measured time at 80%, 90%, 95% and 99% recall after each method chose its own settings. No tested B or C setting beat that choice at the same target.
2. A and B were close at lower targets. This does not mean A is always best.
3. The successful B/C settings still scored the same documents as A. More selective settings either took longer or missed too many reference IDs.
4. Keep documents and queries unchanged, allow each method its own settings, and compare at the same recall and RAM limits. Extra index work must cost less than the scores it avoids.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.